Post-Cleanup Debug and Instrumentation Architecture

Datalog emission, DWARF, ethdebug, gas tracing, and crypto-safety opportunities

Implementor-agent directive - 2026-05-24

Purpose: Guide the next clean phased push after the current origin/trace cleanup lands. The document specifies how to add Datalog emission, DWARF, ethdebug, gas/source attribution, and forward-looking cryptographic application instrumentation without reintroducing SSOT violations.

Primary constraint: Datalog, DWARF, ethdebug, LSP, CLI, browser views, and agent tools are downstream consumers. They must never define compiler identity or repair incomplete compiler truth.

Document status

- This is an implementation directive, not a merge-ready spec.
- It assumes the current cleanup round keeps phase-owned origin identity, OriginExportKey, trace-facts, fixture/prototype labeling, and trace validation intact.
- Where the current compiler cannot emit a fact yet, the correct output is unknown, ambiguous, synthetic, or fixture-backed - not a fabricated source mapping.

Contents

1. Executive direction

After the current cleanup round, the next architecture should treat Datalog, DWARF, ethdebug, gas tracing, LSP, CLI, and browser graph views as projections over the same canonical trace substrate. The goal is not to build three new debug systems; it is to let several export/query tools consume one validated compiler trace.

```
Phase-owned compiler state
  HIR / MIR / codegen / backend
      ->
Canonical trace facts
  typed facts using OriginExportKey
      ->
Trace validation
  endpoint checks, kind checks, no raw IDs
      ->
Derived debug/analysis model
  source attribution, variables, locations, code objects, scopes
      ->
Export/query adapters
  Datalog relations
  DWARF sections
  ethdebug JSON
  LSP / CLI / HTTP reports
```

Core rule: Datalog, DWARF, and ethdebug are consumers. They must not define compiler identity, patch missing compiler origin data, or become the only place where a compiler fact exists.

The immediate strategic recommendation is to prioritize a small DebugBundle builder and Datalog base emission before deep DWARF or ethdebug work. The DebugBundle gives all downstream exporters one derived attribution policy. Datalog gives agents and developers explainable query power. ethdebug should be prioritized over DWARF for EVM/smart-contract targets; DWARF is still valuable for native/RISC-V experiments and general debugging interoperability.

2. Preconditions and non-negotiable SSOT rules

Start this push only after the current cleanup round has locked the authority boundaries. If these conditions are not true, do not add DWARF or ethdebug yet; the new exporters will freeze the wrong architecture.

1. common::origin owns only generic identity/export primitives.
2. HIR, MIR, codegen, and backend own phase-local origin types and phase-specific origin semantics.
3. trace-facts owns typed serializable fact structs and validation.

4. common::facts is removed, quarantined, or explicitly transitional.
5. The Fibonacci diagnostic remains clearly labeled fixture-backed until real compiler fact emission replaces it.
6. Trace validation rejects missing endpoints, invalid keys, duplicate conflicting records, and malformed free-form fields.
7. Source spans are display metadata, not identity.
8. No report, relation table, debug map, source map, or browser view is the only place where compiler truth exists.

Layer	May own	Must not own
common	Generic origin/export key mechanics; common source-span representation if truly generic	HIR/MIR/Sonatina/bytecode semantics; universal live OriginNode enum
Phase crates	Phase-local identity, lowering decisions, storage decisions, compiler events	Downstream Datalog/DWARF/ethdebug schemas
trace-facts	Typed serializable emitted facts; validation rules	Compiler semantics; derived query conclusions
trace-query / Datalog	Derived relations, report queries, rule versions	Base compiler facts; canonical identity
debug-model	Derived DebugBundle view over trace facts	Phase internals or non-recomputable truth
DWARF / ethdebug exporters	Format-specific serialization	Compiler identity or origin attribution policy
LSP/CLI/HTTP	Presentation, interaction, agent probing	The only copy of any compiler fact

3. Target architecture and authority boundaries

3.1 One-way data flow

The data flow must remain one-way for the first implementation wave. Query results may guide developers and agents, but they do not feed back into normal compilation.

```
compiler phases -> trace-facts -> validation -> debug-model -> query/export -> reports/views


Allowed:
 fe compile --emit-trace -> trace.jsonl
 fe trace query trace.jsonl gas-by-source
 fe debug emit --format ethdebug --input trace.jsonl


Not allowed in this push:
 Datalog result -> changes compiler lowering
 ethdebug exporter -> reaches into HIR/MIR internals directly
 LSP hover -> constructs ad hoc origin identity from names or spans
```

3.2 Authority zones

- HIR owns
  - HirExprOrigin, HirStmtOrigin, HirLocalOrigin, HIR-to-source facts, HIR export projection.
- MIR/runtime owns
  - Runtime instance identity, MIR statement/terminator/local identity, HIR-to-MIR lowering links, MIR storage decisions, runtime-local facts.
- Codegen/backend owns
  - Sonatina and bytecode/asm instruction identity, backend synthetic/legalization events, stack/register/storage facts, instruction categories when emitted by the backend.
- Trace facts owns
  - Serialization and validation of emitted facts. It does not interpret compiler semantics.
- Debug model owns
  - A recomputable view: primary source attribution, all-origin candidates, scope/type/location tables, confidence and attribution policy versions.
- Datalog/query owns
  - Derived relations over base facts. Rule packs are versioned and read-only.
- DWARF/ethdebug own
  - Format-specific projection only. They never own identity.

4. Derived DebugBundle model

DWARF and ethdebug should not independently walk trace facts and invent different attribution policies. Add a small derived debug model, called DebugBundle here. It is not compiler truth; it is a recomputable view over validated trace facts.

```
pub struct DebugBundle {
    pub trace_hash: TraceHash,
    pub compiler: CompilerInfo,
    pub sources: Vec<DebugSourceFile>,
    pub code_objects: Vec<DebugCodeObject>,
    pub functions: Vec<DebugFunction>,
    pub scopes: Vec<DebugScope>,
    pub variables: Vec<DebugVariable>,
    pub types: Vec<DebugType>,
    pub instructions: Vec<DebugInstruction>,
```

```
    pub locations: Vec<DebugLocationRange>,
    pub gas: Vec<DebugGasRecord>,
    pub attribution_policy: AttributionPolicyVersion,
}

pub struct DebugInstruction {
    pub key: OriginExportKey,
    pub code_object: OriginExportKey,
    pub pc_range: PcRange,
    pub opcode_or_mnemonic: String,
    pub primary_source: Option<OriginExportKey>,
    pub all_origins: Vec<OriginExportKey>,
    pub classification: InstructionClassification,
    pub confidence: AttributionConfidence,
}
```

4.1 Why DebugBundle is necessary

- DWARF wants compile units, line tables, subprograms, lexical scopes, variables, types, and location lists.
- ethdebug wants source materials, programs over annotated bytecode, high-level types, EVM data pointers, and compilation metadata. The ethdebug spec is still in a design phase and uses JSON Schema draft 2020-12 [1].
- Datalog wants normalized relations and recursive queries.
- LSP/CLI/browser views want responsive summaries, hovers, inlays, graph navigation, and agent-friendly explain APIs.
- Without a shared derived model, each exporter will duplicate origin traversal, source attribution policy, and variable-location policy.

4.2 Required attribution policies

Policy	Meaning	Default use
primary-source-v1	Choose one primary source origin for a final instruction when possible.	Line tables, simple hovers, inlays
inclusive-source-v1	Include all source ancestors through the origin graph.	Graph views, explain commands
synthetic-overhead-v1	Separate compiler-generated work while linking it to a source cause.	Gas/cost reports, Sean-style loop diagnostics
ambiguous-v1	Preserve multiple candidates when optimization destroyed a one-to-one mapping.	Optimized debug exports, agent warnings

Do not overclaim: A line-table row can show one source location, but the trace sidecar should preserve all origin candidates, synthetic reasons, and confidence. Optimized code will often be many-to-one, one-to-many,

duplicated, or merged.

5. Trace fact additions

Add fact families gradually and only when a report/export needs them. Do not add a generic attribute bag first. Keep typed facts as the schema SSOT.

5.1 Existing core facts

```
TraceFact::OriginNode
TraceFact::OriginEdge
TraceFact::CompilerEvent
TraceFact::Storage
TraceFact::Instruction
TraceFact::InstructionCategory
TraceFact::LoopMembership
TraceFact::InlineContext
```

5.2 Add next

These are the minimum additions needed for Datalog emission, DWARF, ethdebug, gas tracing, and editor integrations.

```
pub struct SourceFileFact {
    pub file_key: OriginExportKey,
    pub uri: String,
    pub display_name: String,
    pub content_hash: String,
    pub source_id: Option<u32>,
}

pub struct SourceSpanFact {
    pub origin: OriginExportKey,
    pub file: OriginExportKey,
    pub start_byte: u32,
    pub end_byte: u32,
    pub start_line: u32,
    pub start_column: u32,
    pub end_line: u32,
    pub end_column: u32,
}
```

```
pub struct CodeObjectFact {
    pub code_object: OriginExportKey,
    pub kind: CodeObjectKind,
    pub owner_function_or_contract: Option<OriginExportKey>,
    pub target: TargetTripleOrEvmVersion,
    pub code_hash: Option<String>,
}

pub struct FunctionFact {
    pub function: OriginExportKey,
    pub name: String,
    pub source_origin: Option<OriginExportKey>,
    pub code_object: Option<OriginExportKey>,
}

pub struct LexicalScopeFact {
    pub scope: OriginExportKey,
    pub parent: Option<OriginExportKey>,
    pub function: OriginExportKey,
    pub source_origin: Option<OriginExportKey>,
}
pub struct TypeFact {
    pub ty: OriginExportKey,
    pub kind: TypeKind,
    pub name: Option<String>,
    pub bit_width: Option<u32>,
    pub fields: Vec<TypeField>,
}

pub struct VariableFact {
    pub variable: OriginExportKey,
    pub name: String,
    pub ty: OriginExportKey,
    pub declaration_origin: OriginExportKey,
    pub scope: Option<OriginExportKey>,
    pub storage_class: VariableStorageClass,
}

pub struct LocationRangeFact {
    pub subject: OriginExportKey,
```

```
    pub code_object: OriginExportKey,
    pub pc_range: PcRange,
    pub location: ValueLocation,
    pub reason: StorageReason,
    pub confidence: LocationConfidence,
}
```

5.3 EVM value locations

```
pub enum ValueLocation {
    SsaValue { value: OriginExportKey },
    StackSlot { offset: i64 },
    Register { name: String },
    EvmStack { depth_from_top: u32 },
    EvmMemory { offset: LocationExpr, length: Option<LocationExpr> },
    EvmStorage { slot: LocationExpr, offset_bits: Option<u16>, width_bits: Option<u16> },
    EvmCalldata { offset: LocationExpr, length: Option<LocationExpr> },
    Unknown { reason: String },
}
```

5.4 Gas facts

```
pub struct StaticGasFact {
    pub instruction: OriginExportKey,
    pub schedule: GasSchedule,
    pub base_cost: u64,
    pub dynamic_cost_kind: Option<DynamicGasKind>,
}

pub struct DynamicGasStepFact {
    pub trace_id: String,
    pub step_index: u64,
    pub code_object: OriginExportKey,
    pub pc: u32,
    pub instruction: Option<OriginExportKey>,
    pub gas_before: u64,
    pub gas_after: u64,
    pub gas_cost: u64,
}
```

Static vs dynamic gas: Static gas is target/schedule dependent and useful for hints. Dynamic gas is transaction/test dependent and better for profiling. Reports and LSP hints must label which one they show.

6. Datalog emission and query design

6.1 Role of Datalog

Datalog should answer graph-shaped, multi-hop questions. It should consume validated base facts and produce derived relations and reports. It should not become a compiler phase.

- Which bytecode instructions originate from this source expression?
- Which source ranges accumulate the most gas?
- Which variables are live at this PC?
- Where did local b first become stack-resident?
- Which zero-extensions are inside a loop?
- Which instructions have no source attribution?
- Which debug ranges are ambiguous after optimization?

6.2 Crate/module layout

```
crates/trace-facts/
  fact.rs
  sink.rs
  jsonl.rs
  validate.rs
  relation.rs       # generated/derived relation view only

crates/trace-query/
  load.rs
  datalog_emit.rs
  rules/
    core.dl
    gas.dl
    debug.dl
    optimizer.dl
  reports/
    gas_by_source.rs
    explain_pc.rs
    variables_at_pc.rs
    loop_cost.rs
```

6.3 Typed facts are the Datalog schema SSOT

Every Datalog base relation should be generated from typed facts. If a derive macro is too much now, implement a relation trait directly beside the fact struct. Do not split relation name, column order, row encoder, docs, and validators across separate registries.

```
pub trait TraceRelation {
    const NAME: &'static str;
```

```
    fn schema() -> RelationSchema;
    fn row(&self) -> RelationRow;
}

// Later:
#[derive(TraceRelation)]
#[trace_relation(name = "origin_edge")]
pub struct OriginEdgeFact {
    #[trace_col(key)]
    pub from: OriginExportKey,
    #[trace_col(key)]
    pub to: OriginExportKey,
    pub label: OriginEdgeLabel,
}
```

6.4 Base relations

```
base_origin_node(node, kind)
base_origin_edge(from, to, label)
base_source_file(file, uri, hash)
base_source_span(origin, file, start_byte, end_byte, start_line, start_col, end_line, end_col)
base_code_object(code_object, kind, target, code_hash)
base_function(function, name, source_origin, code_object)
base_scope(scope, parent, function, source_origin)
base_variable(variable, name, type, declaration_origin, scope, storage_class)
base_type(type, kind, name, bit_width)
base_instruction(instruction, code_object, pc_start, pc_end, opcode)
base_instruction_category(instruction, category, source)
base_storage(subject, phase, location, reason)
base_location_range(subject, code_object, pc_start, pc_end, location, reason, confidence)
base_event(event, phase, kind, reason)
base_event_input(event, input)
base_event_output(event, output)
base_loop_member(loop, instruction)
base_inline_context(inline_instance, caller, callee, callsite)
base_static_gas(instruction, schedule, base_cost, dynamic_kind)
base_dynamic_gas(trace_id, step, code_object, pc, instruction, gas_cost)
```

6.5 Core derived relations

```
origin_reaches(A, B) :-
    base_origin_edge(A, B, _).
```

```
origin_reaches(A, C) :-
  base_origin_edge(A, B, _),
  origin_reaches(B, C).

instruction_source(Inst, Src) :-
  base_instruction(Inst, _, _, _, _),
  origin_reaches(Inst, Src),
  base_source_span(Src, _, _, _, _, _, _, _).

unmapped_instruction(Inst) :-
  base_instruction(Inst, _, _, _, _),
  not instruction_source(Inst, _).

variable_at_pc(Var, CodeObject, Pc, Location) :-
  base_variable(Var, _, _, _, _, _),
  base_location_range(Var, CodeObject, Start, End, Location, _, _),
  Start <= Pc,
  Pc < End.
```

6.6 CLI shape

```
fe trace validate target/fib.trace.jsonl

fe trace export-datalog  --input target/fib.trace.jsonl  --out target/fib.datalog/

fe trace query  --input target/fib.trace.jsonl  --rules gas-v1  gas-by-source

fe trace query  --input target/fib.trace.jsonl  variables-at-pc --pc 42

fe trace query  --input target/fib.trace.jsonl  explain-pc --pc 42
```

6.7 Datalog acceptance criteria

9. Typed facts generate base relations.
10. Relation schemas are not hand-mirrored.
11. trace validate runs before export.
12. origin_reaches works over fixture and real traces.
13. explain-pc works over at least one trace.
14. gas-by-source works over static gas facts.
15. dynamic-gas-by-source works over imported execution traces.
16. Missing or ambiguous mappings are reported explicitly.

7. DWARF design

DWARF is the right export path for native-ish artifacts, RISC-V/ELF experiments, external debugger integration, and general source-level debug interoperability. For EVM smart contracts, prefer ethdebug as the first-class debug format. DWARF Version 5 is the sensible initial target; the DWARF site lists v5 as the current published standard and describes it as an upward-compatible extension of v4 with improved line number program support and other changes [5].

7.1 Use gimli::write

Use gimli::write unless there is a compelling reason not to. The gimli write module is designed to write DWARF debugging information by building an in-memory representation and then writing sections [6].

```
crates/debug-export/
  src/lib.rs
  src/model.rs        # shared DebugBundle adapter, if not separate crate
  src/dwarf/
    mod.rs
    line.rs
    info.rs
    types.rs
    locations.rs
    validate.rs
```

7.2 DWARF MVP: line tables only

- Input: DebugInstruction rows with code object, PC range, primary source span, classification, and confidence.
- Output: PC/address range to file/line/column mapping.
- Constraint: one primary source range per instruction for the line table, while the trace sidecar preserves all candidates.
- Synthetic/unmapped instructions are omitted from the line table or mapped to a clearly compiler-generated location; they must not be faked as user source.

Trace/debug model	DWARF output
SourceFileFact	Line-table file entry
CodeObjectFact	Compilation unit/code range
FunctionFact	Subprogram range, later
DebugInstruction	Line-table rows
SourceSpanFact	File/line/column attributes

7.3 DWARF phase 2: compile units and subprograms

- Map Fe package/module/source file to DW_TAG_compile_unit.
- Map Fe function or contract method to DW_TAG_subprogram.
- Map lexical scopes to DW_TAG_lexical_block.

- Use a conservative language attribute strategy until Fe has an assigned DWARF language code; vendor metadata may be appropriate during experimentation.

7.4 DWARF phase 3: types and variables

- TypeFact -> DW_TAG_base_type, DW_TAG_structure_type, DW_TAG_array_type, DW_TAG_member, and related type DIEs.
- VariableFact -> DW_TAG_variable and DW_TAG_formal_parameter.
- LocationRangeFact -> location lists.
- For native/RISC-V targets: register locations, frame base offsets, and stack slots can become standard DWARF location expressions.
- For EVM: do not fake stack/memory/storage/calldata as native registers unless clearly marked experimental; use ethdebug pointers for EVM-native debugging.

7.5 DWARF phase 4: inline call stacks

Once InlineContextFact is real, emit DW_TAG_inlined_subroutine. This matters for Sean-style cases where a final instruction belongs to fib, inlined into main at a callsite. The DebugBundle must preserve both callee source and callsite source.

7.6 DWARF tests

- Generate DWARF line table for a tiny Fe function.
- Verify line rows map PCs to expected source lines.
- Run llvm-dwarfdump/readelf smoke tests in CI if available.
- Ensure unmapped/synthetic instructions do not get fake source lines.
- Ensure same local IDs in different owners do not collapse.
- Golden test for inlined function once inline facts exist.
- Golden test for variable location list once storage facts exist.

8. ethdebug design

ethdebug is the right target for smart-contract debugging. The ethdebug overview describes schemas for high-level data types, EVM data pointers, programs over annotated compiled bytecode, data representations, and source materials; the schemas are JSON Schema draft 2020-12 [1]. Solidity has already started experimental support: Solidity 0.8.29 introduced ethdebug output for instructions and source ranges, limited to unoptimized compilation via IR and missing many important features [2].

Solidity has also identified the central optimized-debugging problem: optimization can duplicate or merge blocks and destroy the simple one-to-one relationship between source and generated instructions, so debug annotations need corresponding transformations [3]. Fe should use this as a warning: optimized debug information must preserve uncertainty rather than fabricate precision.

8.1 ethdebug exporter role

```
validated TraceFacts
 -> DebugBundle
 -> ethdebug JSON
```

```
-> optional Fe trace sidecar / origin index
```

8.2 ethdebug MVP: materials, program, and instruction source ranges

- Emit source materials/source inputs.
- Emit runtime bytecode program; creation bytecode can follow.
- Emit instruction list with bytecode offsets/PCs.
- Attach source ranges where primary-source attribution is confident.
- Preserve OriginExportKey and richer classification in a sidecar if the ethdebug schema does not carry it natively.

Trace/debug model	ethdebug output
SourceFileFact	materials/source input
CodeObjectFact	program
InstructionFact	instruction annotation
SourceSpanFact	source range
OriginExportKey	sidecar or extension metadata
CompilerInfo	info/metadata

8.3 ethdebug phase 2: type schema

- Emit high-level Fe types from TypeFact.
- Start with bool, unsigned/signed integers, address, unit, tuple, array, struct, contract reference.
- Type identity should be TypeFact.ty / OriginExportKey, not just a display name.

8.4 ethdebug phase 3: pointer/location schema

The pointer schema is a major unlock: it lets a debugger know where high-level variables live in EVM state. Map LocationRangeFact into EVM stack, memory, storage, and calldata pointer records.

- EvmStack(depth) -> stack pointer.
- EvmMemory(offset, len) -> memory pointer.
- EvmStorage(slot, offset, width) -> storage pointer.
- EvmCalldata(offset, len) -> calldata pointer.
- Unknown(reason) -> absent, ambiguous, or explicit unknown depending on schema support.

8.5 ethdebug phase 4: dynamic execution and gas

```
EVM execution trace
 step PC, gas_before, gas_after, opcode
      ->
join with InstructionFact / ethdebug instruction
      ->
join with OriginExportKey / source attribution
      ->
gas-by-source / gas-by-variable / gas-by-loop reports
```

8.6 ethdebug phase 5: optimized code

- Every instruction is directly attributed, synthetic with reason, optimizer-created, snapshot-preserved, backend-prepared, unmapped with reason, or ambiguous with candidates.
- If ethdebug cannot represent the full reason/candidate set, keep it in Fe trace sidecar.
- Do not coerce optimized instructions into fake source mappings.

8.7 ethdebug tests

- Validate JSON against a pinned ethdebug schema version.
- Runtime bytecode instruction count matches disassembly.
- Every source range points at a known source material.
- Every instruction has source, synthetic, ambiguous, or unmapped classification.
- For simple locals, pointer location resolves against simulated EVM state.
- For storage variables, slot/offset mapping is correct.
- Optimized builds preserve explicit unknown/ambiguous classifications.

9. Gas tracing to syntax

Gas-to-source attribution is one of the highest-value developer experiences for a smart-contract-oriented Fe toolchain. Solidity source maps already map bytecode instructions to source ranges for debugging and breakpoint handling, including fields such as jump type and modifier depth [4]. Fe should treat legacy source maps as a compatibility view, while trace facts and DebugBundle carry richer identity, synthetic reasons, and confidence.

9.1 Static gas attribution

- Inputs: InstructionFact, StaticGasFact, OriginEdgeFact, SourceSpanFact, LoopMembershipFact.
- Derive: instruction -> primary source; source range -> static gas sum; loop -> static per-iteration estimate; function -> static gas shape.
- Use: LSP inlays, quick hovers, CI budget warnings, and agent triage.
- Caveat: static gas is not transaction gas; label target and gas schedule.

9.2 Dynamic gas attribution

- Inputs: EVM execution trace, DynamicGasStepFact, InstructionFact, OriginEdgeFact, SourceSpanFact.
- Derive: execution step -> instruction -> source; source -> actual gas in this run; loop -> iteration count and gas; function/call -> dynamic cost.
- Use: test-specific profiling, browser timeline, agent explanations for expensive transactions, source-level gas flamegraphs.
- Caveat: dynamic gas is scenario-specific; never present it as universal.

9.3 Attribution policies

Policy	Behavior	Good for
exclusive-primary	Each instruction charged to one primary source origin.	Inlays, concise reports
inclusive	Cost shown on every relevant source ancestor.	Graph exploration, broad call/source views
synthetic-overhead	Compiler-generated work	Optimization diagnostics

	charged separately but linked to source cause.	
call-inclusive	Call source includes callee cost.	User-facing gas by entrypoint
call-exclusive	Call source excludes callee body.	Compiler-level attribution

Policy labeling: Every gas report must name its attribution policy and confidence. This avoids double-counting confusion and helps agents avoid overconfident recommendations.

10. LSP, CLI, HTTP, and agent integration

10.1 Reuse the same query/export stack

The LSP should not run its own compiler archaeology. It should call the same trace-query APIs and DebugBundle builder that the CLI and browser use. The CLI is the automation-friendly surface; LSP is the contextual surface; HTTP/browser views are the graph-rich surface; agents should be able to use all three.

10.2 Inlay hints

- ~N static gas or +N dynamic gas from last selected test run.
- loop: N instructions/iteration or N gas/iteration.
- stack slot / EVM storage / calldata pointer marker for variables.
- zext x2 / normalization overhead markers for compiler-generated work.
- unmapped / ambiguous debug mapping marker.

10.3 Hover details

- Origin chain and primary source attribution.
- Gas breakdown: static, dynamic, synthetic overhead, call-inclusive/call-exclusive.
- Variable location at PC: EVM stack/memory/storage/calldata pointer or native register/stack slot.
- Why an instruction exists: user computation, integer legalization, spill/reload, backend-prepared, optimizer-created.
- Debug confidence and missing/ambiguous mapping reasons.

10.4 CLI commands

```
fe trace emit --target evm --debug-info trace-jsonl

fe trace query gas-by-source --input target/foo.trace.jsonl

fe debug emit --format ethdebug   --input target/foo.trace.jsonl   --out target/foo.ethdebug.json

fe debug emit --format dwarf   --input target/foo.trace.jsonl   --out target/foo.debug.o

fe debug validate --format ethdebug target/foo.ethdebug.json
```

```
fe debug explain-pc --pc 42 --input target/foo.trace.jsonl
```

10.5 Agent-friendly HTTP/browser flow

- Expose read-only endpoints for trace manifest, source files, origin graph slices, instruction/source attribution, variables-at-PC, gas-by-source, and explain-PC.
- Use OriginExportKey as the graph node ID in the browser; never use display names as join keys.
- Make graph slices lazy: source node -> HIR/MIR/codegen -> bytecode -> gas/trace steps.
- Include report provenance: input trace hash, rule versions, compiler version, target, gas schedule, and confidence.

11. Implementation phases and acceptance gates

Phase	Deliverables	Acceptance gate
0. Finish cleanup guardrails	Retire/quarantine common::facts; harden trace validation; fixture status explicit; canonical SourceFile/SourceSpan facts.	No downstream layer constructs identity from raw strings or source spans.
1. DebugBundle MVP	Builder from validated TraceFacts; source table; code object table; instruction table; primary-source policy; confidence model.	Given a trace, produce instruction -> source mappings with explicit confidence and unmapped/synthetic preservation.
2. Datalog base emission	TraceRelation trait/derive; base relation export; schema manifest; core origin_reaches rules.	fe trace export-datalog works on fixture and real traces; no hand-mirrored relation schemas.
3. First query reports	explain-pc; variables-at-pc; gas-by-source static; loop-cost using query layer.	Sean-style report can be generated from facts, not hard-coded report text.
4. DWARF line-table MVP	DWARF v5 line table; minimal compilation unit; PC/source rows; smoke tests.	Native/RISC-V artifact line mappings match source spans.
5. DWARF functions/scopes/types/vars	Subprograms, lexical blocks, basic types, variables, location lists, inline subroutine support.	Simple local variable can be located over a PC range for a non-EVM artifact.
6. ethdebug instruction/source MVP	materials; runtime bytecode program; instruction/source annotations; schema validation; sidecar origin index.	Bytecode PC maps to Fe source range or explicit synthetic/unmapped/ambiguous classification.
7. ethdebug types/pointers	Type export; variables; stack/memory/storage/calldata pointers.	Debugger can identify a Fe local/state variable and describe how to read it from EVM state.
8. Dynamic gas tracing	EVM trace ingestion; DynamicGasStepFact; PC -> instruction -> source join; LSP inlay prototype.	Given a test transaction, Fe shows source-level dynamic gas attribution.
9. Optimized-code honesty	Optimizer classifications;	Optimized exports do not

	ambiguous/multi-origin records; synthetic overhead reports.	pretend precision the compiler did not record.

12. Testing and validation strategy

12.1 Trace validation

- Every fact key is a canonical OriginExportKey.
- Every origin edge endpoint exists.
- Every storage subject exists.
- Every instruction belongs to exactly one code object/function owner.
- Every instruction category references a known instruction.
- Loop membership references a known loop and known instruction.
- Inline context endpoints exist and have plausible kinds.
- No raw local IDs appear in serialized facts.
- No free-form entity IDs like "b" or "stmt_7" appear as identity.
- Derived facts are marked as derived and include ruleset version.

12.2 Serialization validation

- Newtypes that validate non-empty or normalized strings must enforce that during Deserialize, not only through constructors.
- Trace JSONL should have a manifest with compiler version, target, gas schedule, source hashes, trace schema version, and build settings.
- DebugBundle should include input trace hash and attribution policy version.
- Derived reports should include input trace hash, query ruleset version, and timestamp/tool version.

12.3 Export validation

- Datalog export validates relation schemas and rows against typed fact definitions.
- DWARF export passes basic dwarfdump/readelf smoke tests where tools are available.
- ethdebug export validates against pinned JSON Schema version.
- Gas reports recompute summaries from rows and fail if summary totals drift.
- LSP hovers/inlays expose confidence and never hide unmapped/ambiguous facts.

13. Risks and anti-patterns

Anti-pattern	Why it is bad	Correct move
Universal live OriginNode enum in common	Centralizes phase semantics and recreates dependency pressure.	Phase crates own identity; exports use OriginExportKey.
Datalog creates base compiler facts	Turns query engine into a second compiler model.	Datalog only derives from compiler-emitted base facts.
DWARF exporter reads HIR/MIR internals	Duplicated attribution policy and crate coupling.	Exporter consumes DebugBundle.
ethdebug source range faked for synthetic code	Misleads debugger and agents.	Emit synthetic/unmapped/ambiguous

		classification.
Gas report without attribution policy	Causes double counting and wrong optimization advice.	Always show policy, target, schedule, confidence.
Source span used as identity	Breaks under inlining, generated code, same-line constructs, macro-like expansion.	Use OriginExportKey; spans are display metadata.
Debug artifacts include private witness values	Leaks secrets in crypto/zk workflows.	Use redacted facts and explicit secret/public policy.
Content hash excludes compiler settings	False reproducibility.	Manifest must hash source, settings, compiler version, target, dependencies, and trace schema.

Appendix A. Full-stack crypto, zk/EVM, content addressing, and safety opportunities

This appendix looks beyond the immediate Datalog/DWARF/ethdebug push. The same debug/instrumentation/content-addressing infrastructure can become a major differentiator for a full-stack target language designed for applied cryptography, smart contracts, and zk/EVM applications. Sean is already working on a full-stack zk/EVM demo; the instrumentation architecture should make that kind of work easier to understand, audit, and trust.

A.1 Why this matters for applied cryptography

Cryptographic applications fail at boundaries: source to circuit, circuit to witness, witness to proof, proof to verifier, verifier to EVM, EVM to storage/events, and deployed artifact to frontend/client code. A full-stack language can reduce those boundary failures if the compiler records how each artifact was produced and lets tools trace behavior back to source.

- A source expression should be traceable to HIR/MIR/codegen, circuit constraints, witness inputs, verifier bytecode, gas costs, storage writes, logs, and deployment metadata where applicable.
- A failed proof or failed constraint should be traceable back to the source expression and input classification that caused it.
- A gas spike in a verifier should be traceable to source constructs, constraint-system choices, hash choices, or pairing/precompile usage.
- A security reviewer should be able to ask whether secret data reaches a public output, storage slot, event log, revert string, calldata return, or unredacted debug artifact.

A.2 Content-addressed audit bundles

Content addressing can turn compiler artifacts into auditable evidence bundles. IPFS CIDs are labels based on content hashes and differ when content differs; CIDv1 also carries self-describing codec/hash information [8]. Fe does not need to depend on IPFS specifically, but it should adopt the same principle: artifacts should be addressed by content and linked by a manifest.

```
AuditBundleManifest {
  source_files: [content hash, uri, OriginExportKey],
```

```
  dependencies: [content hash, version, registry/source],
  compiler: [version, commit, build flags],
  settings: [target, optimizer, gas schedule, circuit backend],
  trace: [trace schema version, trace hash],
  debug: [DebugBundle hash, attribution policy],
  bytecode: [creation hash, runtime hash],
  ethdebug: [schema version, artifact hash],
  dwarf: [sections hash, target],
  circuit: [constraint system hash, proving system, backend],
  proving_keys: [hashes, generation settings],
  verifier: [source hash, bytecode hash, deployment metadata],
  tests: [dynamic traces, gas traces, proof traces],
  signatures: [optional producer/reviewer attestations]
}
```

- Reproducibility: rebuild and verify artifact hashes match.
- Audit diffing: identify exactly what source, settings, constraints, keys, or bytecode changed.
- Agent workflows: let an agent load a bundle and ask questions without scraping ad hoc files.
- Deployment safety: compare deployed bytecode and verifier keys against the audited bundle.

Caution: A content hash proves content identity, not correctness. The manifest must include compiler version, settings, dependencies, target, and schema versions; otherwise two builds may appear comparable when they are not.

A.3 Source-to-circuit-to-proof-to-EVM traceability

For a full-stack zk/EVM demo, add domain-specific trace facts that connect the same OriginExportKey spine across the whole pipeline.

Domain	Facts to add	Questions enabled
Circuit IR	CircuitNodeFact, ConstraintFact, GateFact, SignalFact, PublicInputFact, PrivateWitnessFact	Which source produced this constraint? Which values are public?
Witness generation	WitnessInputFact, WitnessAssignmentFact, WitnessRedactionPolicy, WitnessFailureFact	Which source input caused a failed constraint? Is a secret being logged?
Proof system	ProofArtifactFact, ProvingKeyFact, VerifyingKeyFact, ProofParameterFact	Which constraints and settings produced this proof/key?
Verifier generation	VerifierFunctionFact, PrecompileUseFact, VerifierGasFact, CalldataLayoutFact	Which circuit components dominate verifier gas?

EVM deployment	DeploymentArtifactFact, ContractAddressFact, CodeHashFact, StorageLayoutFact	Does deployed bytecode match the audited verifier?
Frontend/client	ClientBindingFact, ABIEncodingFact, InputValidationFact	Does the frontend send the right public inputs and calldata?

A.4 Constraint and gas attribution

The gas-to-source story generalizes to constraint-to-source attribution. For zk applications, developers need to know not only what is expensive on-chain, but also what is expensive to prove.

- Constraint count by source expression, function, loop, or library call.
- Gate/row usage by primitive: hash, signature check, range check, equality, lookup, pairing/precompile, storage proof.
- Witness generation cost by source region.
- Verifier gas by circuit component and source origin.
- Static estimates and dynamic/prover-run measurements, both with explicit attribution policies.

```
Example report:
  source: verify_membership(path, leaf)
  constraints: 12,544
  proving time sample: 184 ms
  verifier gas contribution: ~43,000 gas
  dominant causes:
    Poseidon hash rounds: 78%
    range checks: 12%
    public input packing: 6%
  trace confidence: compiler-emitted + backend measurement
```

A.5 Secret/public taint and debug redaction

A full-stack crypto language needs privacy-aware instrumentation. Debugging tools must help find leaks without becoming leaks themselves.

- Track labels such as secret/private witness, public input, calldata, storage, event, return value, revert string, log, off-chain-only, and declassified.
- Emit taint flow facts from phase-owned analyses; derive leak candidates in Datalog.
- Make redaction policy part of DebugBundle and trace export.
- For LSP and HTTP views, default to redacted witness values; require explicit opt-in for local-only secret inspection.
- Flag flows from private witness to public output, storage, event logs, error strings, or unredacted debug artifacts unless explicitly declassified.

A.6 Constant-time and side-channel-oriented instrumentation

Some cryptographic code must avoid data-dependent timing, gas, memory access, or branching. Fe can use trace facts to make these properties visible, even before it has full formal verification.

- Control-flow dependency facts: branch condition depends on secret? yes/no/unknown.
- Memory/storage access dependency facts: address/slot depends on secret? yes/no/unknown.
- Gas dependency facts: dynamic gas cost depends on secret-labeled value? yes/no/unknown.
- Loop bound facts: iteration count depends on secret? yes/no/unknown.
- Primitive policy facts: approved hash/signature/KEM primitive, domain separation tag, nonce/rng source, curve/modulus compatibility.

NIST approved FIPS 203, 204, and 205 for post-quantum cryptography in 2024, covering ML-KEM, ML-DSA, and SLH-DSA [7]. Fe does not need to implement those immediately, but the language/tooling should be ready for versioned cryptographic primitive policies, algorithm agility, and migration guidance.

A.7 Formal specification and proof-obligation workflow

The origin/trace model can connect source-level specs, generated proof obligations, solver/prover results, and deployed code. This matters for crypto and smart contracts because correctness is often a property of the whole workflow, not only the source file.

- SpecOriginFact: source-level invariant, precondition, postcondition, or protocol assumption.
- ObligationFact: generated verification condition or circuit assertion linked to spec and source origin.
- ProofResultFact: discharged/failed/unknown with solver/prover metadata.
- CounterexampleFact: redacted or sanitized counterexample linked to source origins.
- DeploymentPolicyFact: blocks deployment if critical obligations are failed or unknown.

A.8 Crypto-aware LSP and agent experiences

Surface	Feature	Example
Inlay	Constraint/gas hint	constraints: 1.2k, verifier gas: ~8.4k
Inlay	Privacy label	private witness -> public output? no
Hover	Origin chain	source -> circuit constraint -> verifier bytecode -> gas trace
Hover	Primitive policy	domain separation tag present; RNG source approved
Code lens	Open proof trace	show failed constraint path
Command	Explain verifier gas	rank source/circuit nodes by verifier gas
Agent tool	Audit bundle query	what changed in proving key and verifier bytecode?
Browser graph	Full-stack artifact graph	source, constraints, proof, verifier, deployment, tx trace

A.9 New IRs worth supporting in rich views

- Source/HIR/MIR: user intent, types, desugaring, storage decisions.

- Circuit IR: signals, constraints, public/private boundaries, gates, rows, lookups.
- Witness IR: assignment generation, input layout, redaction boundaries, failure paths.
- Verifier IR: generated verifier logic, precompile calls, calldata packing, pairing checks.
- EVM IR/bytecode: opcode-level gas, storage/memory/calldata pointers, source mapping, ethdebug.
- Deployment IR: constructor args, deployed bytecode, code hash, contract address, chain ID, settings.
- Frontend/client bindings: ABI, input validation, serialization, wallet/chain interactions.

Could it be worth it? Yes, if implemented incrementally. Rich IR views are expensive if each view invents its own model. They are valuable if every view is a projection over the same OriginExportKey and content-addressed trace bundle.

A.10 What not to overpromise

- Instrumentation is not a proof of cryptographic security.
- Content addressing is not a supply-chain security system unless manifests are complete and signed/attested where needed.
- Gas or constraint attribution is an estimate unless derived from a specific dynamic trace or prover run.
- Datalog-derived leak warnings are not a substitute for formal noninterference proofs.
- Debug artifacts can leak secrets unless redaction policy is enforced by default.

Appendix B. Minimal demonstrations for Sean-style zk/EVM work

B.1 Demo 1: content-addressed audit bundle

- Compile a small zk/EVM demo and emit source hashes, trace hash, bytecode hash, ethdebug hash, circuit hash, proving/verifying key hashes, and deployment metadata.
- Command: fe bundle verify target/demo.fe-bundle.json.
- Agent query: what changed between bundle A and bundle B?
- Acceptance: a one-byte source change, optimizer setting change, or proving-key change alters the manifest in a localized, explainable way.

B.2 Demo 2: failed constraint to source

- Use a small circuit with a deliberate failed equality/range check.
- Emit ConstraintFact, SignalFact, WitnessFailureFact, and SourceSpanFact.
- LSP hover on the failing source expression shows the failed constraint path and redacted witness slots.
- Acceptance: no raw witness values shown unless explicit unsafe local mode is enabled.

B.3 Demo 3: verifier gas to source/circuit component

- Compile a verifier and run a dynamic transaction trace.
- Join PC -> instruction -> verifier source -> circuit component -> original source origin.
- Report gas by source construct and circuit component.
- Acceptance: at least one expensive component is explained as hash rounds, pairing/precompile, public input packing, or storage/calldata handling.

B.4 Demo 4: secret/public leak lint

- Mark a value as private witness and attempt to log, return, store, or use it in a data-dependent public branch.
- Emit taint facts and derive leak candidates in Datalog.
- LSP inlay shows privacy label; hover explains the leak path.
- Acceptance: every warning includes an origin path and a suggested policy decision: remove, declassify, hash/commit, or keep private.

B.5 Demo 5: full-stack graph view

- Open a browser graph from source expression to HIR/MIR, circuit constraints, verifier bytecode, ethdebug instruction, gas trace step, and deployed code hash.
- Graph nodes use OriginExportKey or content-addressed artifact IDs.
- Agent tool can request the same graph slice via HTTP.
- Acceptance: graph is lazy and queryable; it does not require loading the entire trace for one hover.

Appendix C. Glossary

Term	Meaning
OriginExportKey	Canonical exported identity for a compiler entity. Downstream facts, debug records, and reports reference this key.
TraceFact	Typed compiler-emitted fact that records identity,

	edges, events, storage, instructions, locations, gas, etc.
DebugBundle	Derived, recomputable debug/export view over validated TraceFacts.
Datalog base relation	Relation generated from emitted TraceFacts.
Datalog derived relation	Relation computed by rules over base facts; not compiler truth.
DWARF	General debugging information format used by native debugger ecosystems.
ethdebug	Smart-contract debugging data format for annotated bytecode, source materials, high-level types, and EVM pointers.
Static gas	Target/schedule-based estimate attached to instructions or source origins.
Dynamic gas	Observed gas from a concrete execution trace.
Content-addressed artifact	Artifact identified by a hash-derived address/ID and linked in a manifest.
Witness	Private or public values used to satisfy a zk circuit. Debugging witness data requires redaction discipline.
Constraint	Circuit-level assertion or relation generated from source/program logic.

Appendix D. Source references

[1] ethdebug format specification overview. https://ethdebug.github.io/format/spec/overview/

[2] Solidity 0.8.29 release announcement, including initial ethdebug support. https://soliditylang.org/blog/2025/03/12/solidity-0.8.29-release-announcement/

[3] The Road to Core Solidity, including ethdebug and optimizer/debug-info discussion. https://www.soliditylang.org/blog/2025/10/21/the-road-to-core-solidity/

[4] Solidity source mappings documentation. https://docs.soliditylang.org/en/latest/internals/source_mappings.html

[5] DWARF Version 5 standard page. https://dwarfstd.org/dwarf5std.html

[6] gimli::write documentation. https://docs.rs/gimli/latest/gimli/write/index.html

[7] NIST announcement approving FIPS 203, 204, and 205 post-quantum cryptography standards. https://www.nist.gov/news-events/news/2024/08/announcing-approval-three-federal-information-processing-standards-fips

[8] IPFS content addressing and CID documentation. https://docs.ipfs.tech/concepts/content-addressing/

Accessed for this directive on 2026-05-24.